be created with the following syntax though there is a limitation with sub-lists that are not fully supported. You need to have a line before each sub-list with a leading tab before it.

```
1.Item1

   1.1 Item 1.1
   1.2 Item 1.2

2.Item2
3.Item3
```

To create simple links, you can simply place a URL or email address inside angle brackets:

```
e.g., <https://www.gnu.org/software/
emacs/> and <bug-gnu-emacs@gnu.org>.
```

Inline style links can be created using Markdown easily with the following syntax:

```
Search freely with [DuckDuckGo]
(https://duckduckgo.com)
```

One can optionally use a title with the following syntax:

```
Search freely with [Duckduckgo]
(https://duckduckgo.com "Title").
```

Reference-style links allow you to refer to your links by names, which you define elsewhere in your document using the following syntax:

Image syntax is very much like link syntax:

```
![alt text](/path/to/img.jpg "Title")
```

There is a reference style for images also:

```
![alt text][id]
[id]: /path/to/img.jpg "Title"
```

Free software editors like GNU Emacs, Vim, and Typora provide support to Markdown.

Codes can be created by wrapping paragraphs in backtick quotes. Any ampersands (&) and angle brackets (< *or* >) will automatically be translated into HTML entities. Code blocks in Markdown are formatted by prefixing each line with four spaces, e.g.:

```
#include <stdio.h>
int main()
{
printf("hello, world\n");
return 0;
}
```

Horizontal rules, corresponding to <*hr*> tags in HTML, are created in Markdown by placing three or more hyphens, asterisks or underscores on a line by themselves.

```
---
- - -
* * *
```

GitHub flavoured Markdown is a dialect of Markdown developed for use on GitHub. It has support for Tasklists, fenced code blocks and Markdown preview.

A sample task list is:

- [ ] Incomplete task
- [x] Completed task

The table of contents can be generated in Markdown by a list of the headings with each ToC entry in the following syntax:

```
• [Introduction](#introduction)
```

…where 'Introduction' is an existing entry in the Markdown document starting with the following:

```
## Introduction.
```

A utility like *markdown-pdf* installed using *npm install markdown-pdf* can be used to convert Markdown to PDF format. Pandoc can be used for conversion from Markdown to various other formats.

reStructuredText

reStructuredText or reST is another documentation language that is part of the Python docutils package developed by David Goodger. It is useful for in-line program documentation (such as Python docstrings), for quickly creating simple Web pages, and for standalone documents. reStructuredText is designed for extensibility in specific application domains. The reStructuredText parser is a component of docutils. reST is used to write technical documentation, books and websites. It is supported, by default, by Emacs with syntax highlighting, or else it can be enabled by typing *M-x rst-mode*. Vi and VSCode also support syntax highlighting for reST.

Retext is a text editor with support for both markdown and reST with live preview functionality. Retext can be installed in Debian based systems with the command *sudo apt -y install retext*. Additional functionality to reST is provided by the Sphinx documentation utility. Sphinx is a tool that makes it easy to create intelligent and beautiful documentation from reStructuredText. The documents written in reST have the extension *.rst* or *.rest*.

When you write documentation in reST, you need to be very precise and stick to certain rules:

- rst syntax, like Python, is sensitive to indentation, so you should be aware when to indent and when not to.
- rst requires blank lines between paragraphs.

Prerequisites

reStructuredText is a technology created by the Python community, so most of the toolchain is built upon Python. It is very important to install Python version 3.4 or above, in advance. docutils can be installed by using the following command in Linux: